Grupparbete — Fångdjupet (textbaserat rollspel, 3 personer)

Typ av examination:	Grupparbete, grupper om 3 personer
Kopplat läranderesultat:	Läranderesultat 8 (Git-arbetsflöde i grupp; läsbar, refaktorerad och testbar kod med enkla designmönster)
Betygsskala:	IG / G
Publiceras:	Vecka 9
Inlämning & redovisning:	Vecka 12
Examinationsform:	Gruppvis, redovisning av Git-historik (PR:ar), kodstruktur och designval

Syfte & mål

Kursens sista uppgift löser ni tillsammans, tre personer i grupp. Fokus ligger inte på att lära nya Java-koncept — ni har redan grunderna, OOP, Collections, Generics, undantag, fil-I/O och JUnit med er från tidigare moment — utan på att arbeta som ett litet utvecklingsteam: feature branches, pull requests, kodgranskning, att lösa en sammanslagningskonflikt tillsammans, och att strukturera koden med enkla designmönster så att den blir läsbar och testbar.

Projektet är tänkt att vara roligt att bygga: ett litet textbaserat rollspel där en hjälte (eller ett gäng hjältar) slåss sig igenom Fångdjupet.

Projektidé: Fångdjupet

Bygg en konsolapplikation där spelaren styr en hjälte genom en serie strider mot fiender i en fängelsehåla. Hjälten har utrustning och egenskaper, fienderna har olika anfallssätt, och framsteg (eller nederlag) kan sparas till fil. Exakt tema och namn får gruppen gärna byta ut mot något egna — poängen är arkitekturen, inte just fantasy-temat.

Föreslagen moduluppdelning (en per gruppmedlem)

Kom överens om gemensamma gränssnitt (interfaces/record) för hjälte, fiende, föremål och händelser innan ni delar upp arbetet — committa det gemensamma "kontraktet" till main tillsammans, skapa sedan varsin feature branch.

- Person A — Strid & karaktärer: Character/Enemy-klasser, hälsopoäng, turordning i strid. Använd Strategy-mönstret för anfallstyper (t.ex. ett AttackStrategy-interface med MeleeAttack, MagicAttack, RangedAttack). JUnit-tester för skadeberäkning.
- Person B — Föremål & inventarium: Item-hierarki (vapen, rustning, drycker), ett Inventory som använder Collections/Generics för att lagra och filtrera föremål. Använd Factory-mönstret för att skapa föremål/fiender utifrån typ och nivå. JUnit-tester för inventariehantering.
- Person C — Värld, spelmotor & sparfunktion: menyloop, sekvens av strider/rum, samt fil-I/O för att spara och läsa in en pågående runda. Använd gärna Observer-mönstret för en händelselogg (t.ex. ett GameEventListener-interface som loggar "Hjälten besegrade Trollet"). JUnit-tester för spara/läsa-logik.

Uppdelningen är ett förslag — gruppen bestämmer själv exakt gränsdragning, men varje medlem ska tydligt äga en modul/paket och synas i Git-historiken genom sina commits och PR:ar.

Exempel på menystruktur

```
Fangdjupet
==========
1. Starta nytt äventyr
2. Fortsätt sparat äventyr
```

```
3. Visa hjältens utrustning
4. Strid mot nästa fiende
e. Avsluta
```

Krav för Godkänt (G)

- En fungerande applikation, byggd i grupp, uppdelad i minst tre tydligt avgränsade paket/moduler — en per medlem.
- Minst två självvalda enkla designmönster används i projektet och kan motiveras muntligt vid redovisningen (t.ex. Strategy, Factory eller Observer).
- Git-arbetsflöde: varje medlem arbetar i egna feature branches och skapar minst två pull requests var under projektets gång (inte en enda stor PR i slutet).
- Kodgranskning: varje pull request ska ha minst en kommentar/granskning från en annan gruppmedlem innan den mergas till main.
- Refaktorerad, läsbar kod: gemensam kodstil i hela projektet, meningsfulla namn och undvikande av tydlig kodduplicering.
- JUnit-tester för den centrala logiken i respektive modul (skadeberäkning, inventariehantering, spara/läsa).
- README.md som innehåller: projektbeskrivning, instruktioner för att köra spelet, vem som ansvarat för vilken modul, vilka designmönster som använts och varför, samt en gruppreflektion kring Git-samarbetet — vad fungerade bra och hur löstes eventuella konflikter.
- Redovisning i vecka 12: gruppen går tillsammans igenom kodstruktur, valda designmönster och Git-historiken (branches, PR:ar och merges).

Frivilliga utmaningar (inget krav, men kul att testa)

- Sätt upp en enkel CI-pipeline med GitHub Actions som kör JUnit-testerna automatiskt vid varje pull request.
- Lägg till fler fiendetyper eller ett "boss-möte" med flera faser och egna anfallsmönster.
- Låt spelet spara högsta poäng/snabbaste genomspelning i en enkel topplista i filen.

Versionshantering & inlämning

- Ett gemensamt Git-repository för hela gruppen (t.ex. på GitHub), där samtliga tre medlemmar är medlemmar/collaborators.
- Arbetet ska synas fördelat över tid och personer i commit-historiken — inte klumpas ihop i en enda persons commits precis innan deadline.
- Lämna in en länk till repositoryt i kursens inlämningssystem senast angivet datum i vecka 12.

Bedömning i korthet

Uppgiften examinerar läranderesultat 8: förmågan att upprätta och använda ett Git-arbetsflöde i grupp (branch, merge, pull request) och att producera läsbar, refaktorerad och testbar kod med stöd av enkla designmönster. Betyget sätts på gruppnivå som IG eller G utifrån hur väl kraven ovan är uppfyllda, men varje medlems individuella bidrag ska gå att följa i Git-historiken.